

CSE 321 Introduction to Algorithm Design Quiz 2 Solution

Question 1

(60 points)

Solve the following recurrence relations by using Master Theorem and give Θ bound for each of them. If any of the relations cannot be solved by using Master Theorem, state that this is indeed the case together with the explanation of the reason.

(a) $T(n) = 2 \cdot T\left(\frac{n}{4}\right) + \sqrt{n \log n}$

(b) $T(n) = 9 \cdot T\left(\frac{n}{3}\right) + 5n^2$

(c) $T(n) = \frac{1}{2} \cdot T\left(\frac{n}{2}\right) + n$

(d) $T(n) = 5 \cdot T\left(\frac{n}{2}\right) + \log n$

(e) $T(n) = 4^n \cdot T\left(\frac{n}{5}\right) + 1$

Solution:

MASTER'S THEOREM

$$T(n) = aT\left(\frac{n}{b}\right) + n^k \log^p n$$

where n is the input size, a is the count of subproblems in the dividing recursive function, and $\frac{n}{b}$ is the size of each subproblem. The constants a, b, k hold the following conditions:

- $a \geq 1$
- $b > 1$
- $k \geq 0$

Master's Theorem states that:

Case 1: If $a > b^k$, then

$$T(n) = \Theta(n^{\log_b a})$$

Case 2: If $a = b^k$, then

a) If $p > -1$, then $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$

b) If $p = -1$, then $T(n) = \Theta(n^{\log_b a} \log \log n)$

c) If $p < -1$, then $T(n) = \Theta(n^{\log_b a})$

Case 3: If $a < b^k$, then

a) If $p \geq 0$, then $T(n) = \Theta(n^k \log^p n)$

b) If $p < 0$, then $T(n) = \Theta(n^k)$

PS: If you didn't learn the Master's theorem as shown above in the class, it is accepted if you ignored $\log n$.

$$(a) \quad T(n) = 2 \cdot T\left(\frac{n}{4}\right) + \sqrt{n \log n}$$

Constants:

- $a = 2 \geq 1$ ✓
- $b = 4 > 1$ ✓
- $k = \frac{1}{2} \geq 0$ ✓

Thus, we can solve this problem by using Master's theorem.

$$b^k = 4^{\frac{1}{2}} = \sqrt{4} = 2 \rightarrow a = b^k \rightarrow \text{Hence, we should follow Case 2.}$$

Since Case 2 has 3 sub-cases, we look into p .

$$p = \frac{1}{2} > -1, \text{ hence we will follow Case 2) a).}$$

Case 2: a) $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
 $T(n) = \Theta(n^{\log_4 2} \log^{\frac{1}{2}+1} n)$
 $T(n) = \Theta(\sqrt{n} \log^{\frac{3}{2}} n)$

$$(b) \quad T(n) = 9 \cdot T\left(\frac{n}{3}\right) + 5n^2$$

Constants:

- $a = 9 \geq 1$ ✓
- $b = 3 > 1$ ✓
- $k = 2 \geq 0$ ✓

Thus, we can solve this problem by using Master's theorem.

$$b^k = 3^2 = 9 \rightarrow a = b^k \rightarrow \text{Hence, we should follow Case 2.}$$

Since Case 2 has 3 sub-cases, we look into p .

$$p = 0 > -1, \text{ hence we will follow Case 2) a).}$$

Case 2: a) $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
 $T(n) = \Theta(n^{\log_3 9} \log n)$
 $T(n) = \Theta(n^2 \log n)$

(c) $T(n) = \frac{1}{2} \cdot T\left(\frac{n}{2}\right) + n$

Constants:

- $a = \frac{1}{2} \not\geq 1$ ✗

Thus, we cannot solve this problem by using Master's theorem.

(d) $T(n) = 5 \cdot T\left(\frac{n}{2}\right) + \log n$

Constants:

- $a = 5 \geq 1$ ✓
- $b = 2 > 1$ ✓
- $k = 0 \geq 0$ ✓

Thus, we can solve this problem by using Master's theorem.

$b^k = 2^0 = 1 \rightarrow a > b^k \rightarrow$ Hence, we should follow Case 1.

Case 1: $T(n) = \Theta(n^{\log_b a})$
 $T(n) = \Theta(n^{\log_2 5})$

(e) $T(n) = 4^n \cdot T\left(\frac{n}{5}\right) + 1$

Constants:

- $a = 4^n \rightarrow a$ is not a constant value. ✗

Thus, we cannot solve this problem by using Master's theorem.

Question 2

(40 points)

Examine the following algorithm and answer the questions.

```
int myFunction2 (int nums[], int n)
{
    int i, *map, max = 0;

    for (i = 0; i < n; i++)
        if (nums[i] > max)
            max = nums[i];

    map = (int *) calloc (max + 1, sizeof (int));

    for (i = 0; i < n; i++)
        map[nums[i]]++;

    for (i = 0; i < n; i++)
        if (map[nums[i]] > n / 2)
            return nums[i];
    return -1;
}
```

- (a) What does the algorithm do? Explain input/output variables.
- (b) What is the time complexity of the algorithm? Analyze the worst and best cases.

Solution:

- (a) The algorithm takes an array of integers and the size of the array as input. Then it uses a map to keep the frequency of each element in the input array. The algorithm uses the index of the map to keep the frequencies and this operation is being done in a single loop. Then it analyzes the map, if there is a value (frequency) larger than $n/2 + 1$, it returns the index of that element (which is the element we are looking for). If there is none, it returns -1.
- (b) Regardless of the best or worst scenario, the algorithm computes the frequency of each element by using a single loop with n iterations. The difference occurs in the last loop. If the first element of the input array has a frequency larger than $n/2 + 1$ this loop would return that element at the first iteration. But since we spent $O(n)$ time just to compute the frequencies, the best time complexity becomes $\Omega(n)$ (The first loop doesn't affect the complexity since it has n iterations as well, not more).

In the worst case, there would not be such an element in the array that holds the frequency condition. So, we complete all loops and spend n time slices for each of them. But since there is no nested loop, we should add the number of iterations (instead of multiplying as in Question 3). Since constant values are ignored, $3n$ time slices project to $O(n)$.

PS: This solution ignores the complexity of the calloc function since the size of the map is unknown. But if we take it into consideration and if the map is too large, then the time complexity could be larger.